

# 2024년 3회 정보처리기사 실기 복원 문제 (해설 추가본)

출처: <https://chobopark.tistory.com/495> (Life-Journey)

## 1번

다음은 Java 코드에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
public class Main {
    static String[] s = new String[3];

    static void func(String[] s, int size){
        for(int i=1; i<size; i++){
            if(s[i-1].equals(s[i])){
                System.out.print("O");
            } else {
                System.out.print("N");
            }
        }
        for (String m : s){
            System.out.print(m);
        }
    }

    public static void main(String[] args){
        s[0] = "A";
        s[1] = "A";
        s[2] = new String("A");
        func(s, 3);
    }
}
```

정답: OOAAA

### 해설:

이 문제의 핵심은 `equals()` 와 `==` 의 차이입니다.

- `equals()` 는 문자열의 **내용(값)**을 비교합니다.
- `==` 는 **주소(참조)**를 비교합니다.

배열에 모두 "A"가 들어있습니다(s[2]는 new String으로 만들어 주소는 다르지만 **내용은 "A"**).

첫 번째 for문은 `equals()` 로 인접한 두 값을 비교합니다:

- s[0]="A" vs s[1]="A" → 내용 같음 → "O"  
- s[1]="A" vs s[2]="A" → 내용 같음 → "O"  
→ 여기까지 "OO"

두 번째 for문은 배열 전체를 출력 → "A","A","A" → "AAA"

합치면 **"OOAAA"**. 만약 `==` 로 비교했다면 s[2]는 주소가 달라 "N"이 나왔을 텐데, `equals()` 를 썼기 때문에 모두 "O"가 된 것이 함정입니다.

## 2번

다음은 파이썬에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
def func(lst):
    for i in range(len(lst) // 2):
        lst[i], lst[-i-1] = lst[-i-1], lst[i]

lst = [1,2,3,4,5,6]
```

```
func(lst)
print(sum(lst[::2]) - sum(lst[1::2]))
```

정답: 3

해설:

이 함수는 리스트를 **앞뒤로 뒤집는(reverse)** 동작을 합니다.

- `lst[i], lst[-i-1] = lst[-i-1], lst[i]` 는 *i*번째와 뒤에서 *i*번째 값을 맞바꾸는 파이썬식 교환입니다.
- `len(lst)//2` = 3번 반복하며 양 끝부터 가운데로 좁혀가며 교환 → 리스트가 완전히 뒤집힙니다.
- `[1,2,3,4,5,6]` → 뒤집으면 **[6,5,4,3,2,1]**

이제 슬라이싱:

- `lst[::2]` 는 인덱스 0,2,4 (짝수 위치) → `[6, 4, 2]`, 합 = 12
- `lst[1::2]` 는 인덱스 1,3,5 (홀수 위치) → `[5, 3, 1]`, 합 = 9
- 결과 = 12 - 9 = **3**

[시작:끝:간격] 슬라이싱에서 간격 2는 "한 칸씩 건너뛰기"를 의미합니다.

### 3번

아래의 employee 테이블과 project 테이블을 참고하여 보기의 SQL 명령어에 알맞는 출력값을 작성하시오. (※ 원문에 테이블 이미지 포함)

```
SELECT count(*)
FROM employee AS e JOIN project AS p ON e.project_id = p.project_id
WHERE p.name IN (
    SELECT name FROM project p WHERE p.project_id IN (
        SELECT project_id FROM employee GROUP BY project_id HAVING count(*) < 2
    )
);
```

정답: 1

해설:

**서브쿼리(쿼리 안의 쿼리)**가 3종으로 중첩된 문제입니다. 가장 안쪽부터 바깥으로 풀어야 합니다.

- 가장 안쪽: `GROUP BY project_id HAVING count(*) < 2` → employee를 프로젝트별로 묶어 **소속 직원이 2명 미만(즉 1명)인 프로젝트 id**를 찾습니다.
- 중간: 그 project\_id에 해당하는 프로젝트 **이름(name)**을 구합니다.
- 바깥: employee와 project를 조인한 뒤, 위에서 찾은 이름에 해당하는 행의 개수를 `count(*)` 로 셉니다.

결과적으로 "직원이 1명만 배정된 프로젝트"에 속한 직원 수를 세는 것이므로 **1**이 나옵니다.

서브쿼리는 안쪽→바깥쪽 순서로 차근차근 결과를 대입해 풀면 됩니다. HAVING은 그룹화 결과에 거는 조건이라는 점을 기억하세요.

### 4번

다음은 운영체제 페이지 순서를 참고하여 할당된 프레임의 수가 3개일 때 LRU 알고리즘의 페이지 부재 횟수를 작성하시오.

페이지 참조 순서: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

정답: 12

해설:

**LRU(Least Recently Used)**는 "**가장 오랫동안 사용되지 않은** 페이지를 먼저 교체"하는 페이지 교체 알고리즘입니다. '페이지 부재(Page Fault)'란 필요한 페이지가 메모리(프레임)에 없어서 새로 가져와야 하는 상황입니다.

프레임 3칸으로 순서대로 따라가 봅시다 (★ = 부재 발생):

|   |           |                 |
|---|-----------|-----------------|
| 7 | → [7]     | ★               |
| 0 | → [7,0]   | ★               |
| 1 | → [7,0,1] | ★               |
| 2 | → [0,1,2] | ★ (가장 오래된 7 교체) |
| 0 | → [1,2,0] | (이미 있음)         |

|   |           |                 |
|---|-----------|-----------------|
| 3 | → [2,0,3] | ★ (가장 오래된 1 교체) |
| 0 | → [2,3,0] | (이미 있음)         |
| 4 | → [3,0,4] | ★ (가장 오래된 2 교체) |
| 2 | → [0,4,2] | ★ (가장 오래된 3 교체) |
| 3 | → [4,2,3] | ★ (가장 오래된 0 교체) |
| 0 | → [2,3,0] | ★ (가장 오래된 4 교체) |
| 3 | → [2,0,3] | (이미 있음)         |
| 2 | → [0,3,2] | (이미 있음)         |
| 1 | → [3,2,1] | ★ (가장 오래된 0 교체) |
| 2 | → [3,1,2] | (이미 있음)         |
| 0 | → [1,2,0] | ★ (가장 오래된 3 교체) |
| 1 | → [2,0,1] | (이미 있음)         |
| 7 | → [0,1,7] | ★ (가장 오래된 2 교체) |
| 0 | → [1,7,0] | (이미 있음)         |
| 1 | → [7,0,1] | (이미 있음)         |

부재 발생 횟수를 세면 총 **12번**입니다.

핵심: 매번 "메모리 안의 페이지 중 가장 오래전에 쓰인 것"을 골라 교체합니다. 프레임이 비어있는 처음에는 무조건 부재가 발생합니다.

## 5번

다음은 네트워크 취약점에 대한 문제이다. 알맞는 용어를 작성하시오.

- IP나 ICMP의 특성을 악용하여 엄청난 양의 데이터를 한 사이트에 집중적으로 보내 네트워크 일부를 불능 상태로 만드는 공격
- 여러 호스트가 특정 대상에게 다량의 ICMP Echo Reply를 보내게 하여 DoS를 유발
- 공격 대상 호스트는 다량 유입 패킷으로 서비스 불능 상태가 됨

정답: 스머프(Smurf) 또는 스머핑(Smurfing)

해설:

스머프(Smurf) 공격은 ICMP(ping)와 브로드캐스트(전체 전송) 특성을 악용한 DoS(서비스 거부) 공격입니다.

동작 원리:

1. 공격자가 출발지 IP를 **피해자의 IP로 위조(스푸핑)**한 ping 요청을 네트워크 전체(브로드캐스트 주소)로 보냅니다.
2. 네트워크의 수많은 호스트가 그 ping에 응답(Echo Reply)하는데, 위조된 출발지인 **피해자에게 한꺼번에 응답이 몰립니다.**
3. 피해자는 폭주하는 응답 패킷 때문에 마비됩니다.

키워드: "ICMP + 브로드캐스트 + 출발지 위조 = 스머프". 비슷한 공격으로 ICMP Flooding, Ping of Death 등이 있으니 "증폭(여러 호스트가 한 명에게 응답)" 특성으로 구분하세요.

## 6번

다음은 GoF 디자인 패턴과 관련된 문제이다. 괄호 안에 알맞는 용어를 작성하시오.

( ) 패턴은 클래스나 객체들이 서로 상호작용하는 방법이나 책임 분배 방법을 정의하는 패턴이다. 객체 간 통신 방법을 정의하고 알고리즘을 캡슐화하여 결합도를 낮춘다. Chain of Responsibility, Command, Observer 패턴이 있다.

정답: 행위

해설:

GoF(Gang of Four) 디자인 패턴은 목적에 따라 **3가지 유형**으로 분류됩니다.

- **생성(Creational):** 객체를 어떻게 만들지에 관한 패턴. 예: Singleton, Factory Method, Abstract Factory, Builder, Prototype.
- **구조(Structural):** 객체들을 어떻게 조합/연결할지에 관한 패턴. 예: Adapter, Bridge, Composite, Decorator, Proxy.
- **행위(Behavioral):** 객체들이 어떻게 **상호작용·책임 분배**를 할지에 관한 패턴. 예: Observer, Strategy, Visitor, Command, Chain of Responsibility, Iterator.

문제에서 "상호작용, 책임 분배, 객체 간 통신, Chain of Responsibility/Command/Observer"라는 키워드가 나오므로 답은 **행위** 패턴입니다.

암기 팁: 생성(만들기), 구조(엮기), 행위(소통하기).

## 7번

다음은 C언어에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>

int func(){
    static int x = 0;
    x += 2;
    return x;
}

int main(){
    int x = 1;
    int sum = 0;
    for(int i=0; i<4; i++) {
        x++;
        sum += func();
    }
    printf("%d", sum);
    return 0;
}
```

정답: 20

해설:

이 문제의 핵심은 **static 지역변수**입니다.

- 일반 지역변수는 함수가 끝나면 사라지지만, **static 변수는 값을 유지**합니다. 함수가 여러 번 호출돼도 **딱 한 번만 초기화**되고, 그 이후로는 이전 값을 기억합니다.

func()이 호출될 때마다 static x는 2씩 누적됩니다:

- 1번째 호출:  $x = 0 + 2 = 2 \rightarrow \text{sum} = 2$
- 2번째 호출:  $x = 2 + 2 = 4 \rightarrow \text{sum} = 6$
- 3번째 호출:  $x = 4 + 2 = 6 \rightarrow \text{sum} = 12$
- 4번째 호출:  $x = 6 + 2 = 8 \rightarrow \text{sum} = 20$

main의 **x++** 는 main의 지역변수 x(이름만 같을 뿐 별개)를 바꾸는 것이라 결과에 영향이 없습니다.

최종  $\text{sum} = 2 + 4 + 6 + 8 = 20$ . "static = 호출 사이에 값을 기억한다"가 핵심입니다.

## 8번

다음은 무결성제약조건에 대한 문제이다. 아래 표에서 어떠한 ( ) 무결성을 위반하였는지 작성하시오. (※ 원문에 표 이미지 포함)

정답: 개체

해설:

**무결성 제약조건**은 데이터베이스의 정확성을 지키기 위한 규칙입니다. 대표적인 세 가지가 있습니다.

- **개체 무결성(Entity Integrity):** 기본 키(Primary Key)는 **NULL이 될 수 없고 중복될 수 없다**는 규칙입니다. 기본 키는 각 행을 유일하게 구분하는 값이므로 비어있거나 겹치면 안 됩니다.
- **참조 무결성(Referential Integrity):** 외래 키는 참조하는 테이블에 실제로 존재하는 값이거나 NULL이어야 한다는 규칙입니다.
- **도메인 무결성(Domain Integrity):** 각 속성 값은 정해진 범위·자료형에 맞아야 한다는 규칙입니다.

이 문제는 기본 키가 비어있거나 중복된 상황이므로 **개체 무결성** 위반입니다. "PK가 NULL이거나 중복 = 개체 무결성 위반"으로 기억하세요.

## 9번

다음은 URL 구조에 관한 문제이다. 아래 보기의 순서대로 URL에 해당하는 번호를 작성하시오. (※ 원문에 URL 이미지 포함)

보기:

- query: 서버에 전달할 추가 데이터

- path: 서버 내의 특정 자원을 가리키는 경로
- scheme: 리소스에 접근하는 방법이나 프로토콜
- authority: 사용자 정보, 호스트명, 포트 번호
- fragment: 특정 문서 내의 위치

정답: 43125

해설:

URL(웹 주소)은 여러 부분으로 구성됩니다. 예시로 보면 이해가 쉽습니다:

```
https://www.site.com:443/folder/page.html?id=10#section2
└scheme┐ └authority┐ └path┐ └query┐ └fragment┐
```

각 부분의 역할:

- **scheme**: 접근 방법/프로토콜 (예: https, ftp) — 맨 앞부분
- **authority**: 사용자 정보 · 호스트명 · 포트 번호 (예: www.site.com:443)
- **path**: 서버 안의 자원 경로 (예: /folder/page.html)
- **query**: ? 뒤에 붙는 추가 데이터 (예: id=10)
- **fragment**: # 뒤의 문서 내 특정 위치 (예: section2)

보기에 매겨진 번호 순서대로 URL의 각 위치에 대응시키면 **43125**가 됩니다. URL의 구조(앞→뒤: scheme → authority → path → query → fragment)를 외워두세요.

10번

다음은 파이썬에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
def func(value):
    if type(value) == type(100):
        return 100
    elif type(value) == type(""):
        return len(value)
    else:
        return 20

a = '100.0'
b = 100.0
c = (100, 200)

print(func(a) + func(b) + func(c))
```

정답: 45

해설:

이 함수는 인자의 **자료형(type)**에 따라 다른 값을 반환합니다.

- type(100) 은 정수형(int)을 의미합니다.
- type("") 은 문자열형(str)을 의미합니다.

각 값을 판정해 봅시다:

- a = '100.0' → 따옴표로 감쌌으니 **문자열**. → len('100.0') = 5글자 = **5**
- b = 100.0 → 소수점이 있으니 **실수(float)**. int도 str도 아니므로 else → **20**
- c = (100, 200) → **튜플(tuple)**. 역시 else → **20**

합계 = 5 + 20 + 20 = **45**

함정: a는 숫자처럼 보이지만 따옴표가 있어 문자열이고, b는 100.0이라 int가 아닌 float입니다. 자료형을 정확히 구분하는 것이 핵심입니다.

11번

다음은 Java 코드에 대한 문제이다. 알맞는 출력값을 작성하시오.

```

public class Main {
    public static void main(String[] args){
        Base a = new Derivate();
        Derivate b = new Derivate();
        System.out.print(a.getX() + a.x + b.getX() + b.x);
    }
}

class Base {
    int x = 3;
    int getX(){
        return x * 2;
    }
}

class Derivate extends Base {
    int x = 7;
    int getX(){
        return x * 3;
    }
}

```

정답: 52

해설:

자바에서 필드(변수)와 메서드의 접근 규칙이 다르다는 점이 핵심입니다.

- 메서드는 오버라이딩 되면 실제 객체 타입 기준으로 호출됩니다. (동적 바인딩)
- 필드(변수)는 선언된 타입(참조 변수의 타입) 기준으로 접근됩니다. (정적 바인딩)

각 항을 봅시다:

- a 는 선언 타입 Base, 실제 객체 Derivate.
- a.getX() → 메서드는 실제 객체(Derivate) 기준 → Derivate의  $getX() = x(7) \times 3 = 21$
- a.x → 필드는 선언 타입(Base) 기준 → Base의  $x = 3$
- b 는 선언 타입도 실제 객체도 Derivate.
- b.getX() → Derivate의  $getX() = 7 \times 3 = 21$
- b.x → Derivate의  $x = 7$

합계 = 21 + 3 + 21 + 7 = 52

핵심 한 줄: "메서드는 실제 객체, 필드는 선언 타입"을 따른다.

## 12번

다음은 C언어에 대한 문제이다. 알맞는 출력값을 작성하시오.

```

#include <stdio.h>

struct Node {
    int value;
    struct Node* next;
};

void func(struct Node* node){
    while(node != NULL && node->next != NULL){
        int t = node->value;
        node->value = node->next->value;
        node->next->value = t;
        node = node->next->next;
    }
}

int main(){
    struct Node n1 = {1, NULL};

```

```
struct Node n2 = {2, NULL};
struct Node n3 = {3, NULL};

n1.next = &n3;
n3.next = &n2;

func(&n1);

struct Node* current = &n1;
while(current != NULL){
    printf("%d", current->value);
    current = current->next;
}
return 0;
}
```

정답: 312

해설:

연결 리스트에서 **두 칸씩 묶어 값을 교환**하는 문제입니다.

먼저 연결 순서를 정리하면: n1(1) → n3(3) → n2(2)  
(n1.next = n3, n3.next = n2)

func은 node 와 node->next 의 **value**를 서로 교환한 뒤, 두 칸 건너뛸니다( node->next->next ).  
- 1회차: node=n1(값1), node->next=n3(값3) → 두 값 교환 → n1=3, n3=1. 그 후 node = n2.  
- 2회차: node=n2이지만 node->next 가 NULL이라 반복 종료.

교환 후 리스트의 값: n1(3) → n3(1) → n2(2)

마지막 출력 루프는 n1부터 따라가며 값을 출력 → 3, 1, 2 → **"312"**

연결 리스트는 **값(value)이 바뀐 것**이지 노드의 연결 순서가 바뀐 게 아니라는 점을 정확히 봐야 합니다.

13번

다음은 테스트 커버리지에 대한 문제이다. 알맞는 답을 보기에서 골라 작성하시오.

- 1. 프로그램의 모든 문장을 최소한 한 번씩 실행했는지를 측정
- 2. 모든 분기(조건문)의 각 분기를 최소한 한 번씩 실행했는지를 측정
- 3. 복합 조건 내의 각 개별 조건이 참과 거짓으로 평가되는 경우를 모두 테스트했는지를 측정

보기: ㄱ. 조건 ㄴ. 경로 ㄷ. 결정 ㄹ. 분기 ㄴ. 함수 ㄷ. 문장 ㄴ. 루프

정답: 1. 문장 / 2. 분기 / 3. 조건

해설:

테스트 커버리지는 "테스트가 코드를 얼마나 꼼꼼히 검사했는가"를 나타내는 척도입니다. 종류별 정의를 구분하는 것이 핵심입니다.

- 1. **문장(Statement) 커버리지**: 코드의 **모든 명령문(줄)**을 최소 한 번씩 실행. 가장 약한 기준입니다.
- 2. **분기(Branch)/결정(Decision) 커버리지**: if문 같은 결정문의 **참/거짓 두 갈래를 모두** 실행. (분기 = 결정으로 같은 의미)
- 3. **조건(Condition) 커버리지**: 결정문 안의 **개별 조건 하나하나**가 참·거짓을 모두 갖도록 테스트. 예: ( A && B ) 에서 A와 B 각각을 참/거짓으로.

강도 순서(약함→강함): **문장 → 분기(결정) → 조건 → 조건/결정 → 변경조건/결정(MC/DC) → 다중조건**. 이 순서는 매우 자주 출제됩니다.

14번

아래는 UML 클래스의 관계에 관한 문제이다. 보기를 보고 알맞는 관계를 선택하여 작성하시오. (※ 원문에 이미지 포함)

보기: ㄱ. 의존 ㄴ. 연관 ㄷ. 일반화

정답: (1) 연관 / (2) 일반화 / (3) 의존

해설:

UML 클래스 다이어그램에서 클래스 간의 관계는 화살표 모양으로 구분합니다.

- **연관(Association):** 두 클래스가 **지속적으로 서로를 알고 사용하는** 관계. 실선으로 표현. 예: 학생과 학교. (선생님이 학생 목록을 멤버로 가지는 등)
- **일반화(Generalization): 상속 관계.** 부모-자식 관계로, 속이 빈 삼각형 화살표로 표현. 예: 동물 ← 개, 고양이.
- **의존(Dependency):** 한 클래스가 **잠깐(일시적으로)** 다른 클래스를 사용하는 관계. 점선 화살표로 표현. 예: 메서드의 매개변수로만 잠깐 쓰는 경우.

구분 팁: 오래 알고 지내면 연관, 부모자식이면 일반화, 잠깐 쓰면 의존. 화살표 모양(실선/점선/삼각형)으로 그림에서 판단합니다.

## 15번

다음은 데이터베이스에 관한 문제이다. 알맞는 답을 보기에서 골라 작성하시오.

- (1) 다른 테이블, 릴레이션의 기본 키를 참조하는 속성 또는 속성들의 집합
- (2) 테이블에서 각 행을 유일하게 식별할 수 있는 최소한의 속성들의 집합
- (3) 후보 키 중에서 선정된 기본 키를 제외한 나머지 후보 키
- (4) 테이블에서 각 행을 유일하게 식별할 수 있는 속성들의 집합

보기: ㉠. 슈퍼키 ㉡. 외래키 ㉢. 대체키 ㉣. 후보키

정답: (1) 외래키 / (2) 후보키 / (3) 대체키 / (4) 슈퍼키

해설:  
데이터베이스의 **키(Key)** 종류를 구분하는 문제입니다. 키는 행(튜플)을 구별하는 식별자입니다.

- **슈퍼키(Super Key):** 각 행을 **유일하게 식별**하는 속성들의 집합. 단, 꼭 필요한 것만 모은 건 아닐 수 있음(불필요한 속성 포함 가능). → (4)
- **후보키(Candidate Key):** 유일하게 식별하면서 **불필요한 속성이 없는 최소한의** 키. 슈퍼키 중 최소 집합. → (2)
- **기본키(Primary Key):** 후보키 중에서 **대표로** 선택한 키.
- **대체키(Alternate Key):** 후보키 중 **기본키로 선택되지 않은 나머지** 키들. → (3)
- **외래키(Foreign Key):** 다른 테이블의 **기본키를 참조**하는 속성. 테이블 간 연결고리. → (1)

포함 관계: 슈퍼키 ㉣ 후보키 ㉡ 기본키. "최소한"이라는 단어가 보이면 후보키를 떠올리세요.

## 16번

다음은 C언어에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>

void func(int** arr, int size){
    for(int i=0; i<size; i++){
        *(*arr + i) = (*(*arr+i) + i) % size;
    }
}

int main(){
    int arr[] = {3, 1, 4, 1, 5};
    int* p = arr;
    int** pp = &p;
    int num = 6;

    func(pp, 5);
    num = arr[2];
    printf("%d", num);
    return 0;
}
```

정답: 1

해설:  
이중 포인터( int\*\* )가 나오지만 실제 핵심은 한 군데, **arr[2]가 어떻게 바뀌는가**입니다.

pp 는 p 를 가리키고, p 는 arr 를 가리킵니다. 따라서 \*arr (함수 안)은 결국 배열 arr를 가리키며, \*(\*arr + i) 는 arr[i] 와 같습니다.

func는 각 원소를 (arr[i] + i) % size 로 바꿉니다(size=5). 우리가 필요한 것은 arr[2]뿐이니 i=2만 계산하면 됩니다.  
- arr[2]의 원래 값 = 4



- 새 값 = (4 + 2) % 5 = 6 % 5 = 1

마지막에 num = arr[2] = 1을 출력합니다.

복잡한 포인터 표기에 겁먹지 말고, `*(arr+i)` 가 `arr[i]` 라는 사실만 알면 i=2 한 줄만 계산하면 됩니다.

## 17번

다음 아래 내용을 보고 알맞는 용어를 작성하시오. (3글자로 작성)

- 공용 네트워크를 통해 사설 네트워크를 확장하는 기술
- 사용자의 IP 주소를 숨기고 추적을 어렵게 만들
- 종류로는 IPsec, SSL, L2TP 등이 있음

정답: VPN

해설:

VPN(Virtual Private Network, 가상 사설망)은 인터넷 같은 공용 네트워크 위에 가상의 안전한 전용 통로를 만드는 기술입니다.

- 회사 밖에서도 마치 사내망에 직접 연결된 것처럼 안전하게 접속할 수 있습니다.
- 통신을 암호화하고 IP를 숨겨 **보안과 익명성**을 높입니다.
- 구현 방식(프로토콜)으로 IPsec, SSL VPN, L2TP 등이 있습니다.

키워드 "공용망 위에 사설망 확장 + IP 숨김 = VPN"으로 기억하세요. 영문 3글자 약자 V-P-N입니다.

## 18번

다음은 Java 코드에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
public class ExceptionHandling {
    public static void main(String[] args) {
        int sum = 0;
        try {
            func();
        } catch (NullPointerException e) {
            sum = sum + 1;
        } catch (Exception e) {
            sum = sum + 10;
        } finally {
            sum = sum + 100;
        }
        System.out.print(sum);
    }

    static void func() throws Exception {
        throw new NullPointerException();
    }
}
```

정답: 101

해설:

자바의 예외 처리(try-catch-finally)를 묻는 문제입니다.

- try 블록에서 예외가 발생하면, 그 예외와 타입이 맞는 첫 번째 catch가 실행됩니다.
- finally 블록은 예외 발생 여부와 상관없이 항상 실행됩니다.

흐름을 따라가면:

- func()이 NullPointerException 을 던집니다(throw).
- catch 절을 위에서부터 확인 → 첫 번째 catch(NullPointerException e) 가 일치 → sum = 0 + 1 = 1.
- 한 번 catch가 잡으면 아래 다른 catch들은 실행되지 않습니다. ( catch(Exception e) 는 건너뛴)
- finally 는 무조건 실행 → sum = 1 + 100 = 101.

핵심: ① catch는 위에서부터 맞는 것 하나만 실행, ② finally는 항상 실행. NullPointerException은 Exception의 자식이라 더 구체적인

NullPointerException 쪽이 먼저 잡힙니다.

## 19번

다음은 Java 코드에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
class Main {
    public static class Collection<T>{
        T value;
        public Collection(T t){
            value = t;
        }
        public void print(){
            new Printer().print(value);
        }
        class Printer{
            void print(Integer a){
                System.out.print("A" + a);
            }
            void print(Object a){
                System.out.print("B" + a);
            }
            void print(Number a){
                System.out.print("C" + a);
            }
        }
    }

    public static void main(String[] args) {
        new Collection<>().print();
    }
}
```

정답: B0

해설:  
제네릭(Generic)과 오버로딩(같은 이름, 다른 매개변수 타입)이 결합된 까다로운 문제입니다.

- new Collection<>() 에서 0이 들어가지만, 제네릭 타입 T 는 컴파일 시점에 정해집니다. 여기서 명시적 타입 지정이 없어 T는 **Object**로 추론됩니다.
- 따라서 value 의 컴파일 시점 타입은 **Object**입니다.
- print(value) 를 호출할 때 자바는 **컴파일 시점 타입(Object)**을 기준으로 어떤 print를 부를지 결정합니다(오버로딩은 정적 결정).
- value의 타입이 Object이므로 print(Object a) 가 선택되어 → "B" + 0 = **"B0"**.

함정: 실제 들어간 값은 정수 0이라 print(Integer) 가 불릴 것 같지만, **변수의 선언(컴파일) 타입이 Object**라서 print(Object)가 호출됩니다. "오버로딩은 컴파일 시점 타입으로 결정된다"가 핵심입니다.

## 20번

다음은 네트워크에 대한 문제이다. 아래 내용을 보고 알맞는 용어를 작성하시오.

- 중앙 관리나 고정된 인프라 없이 임시로 구성되는 네트워크
- 무선 통신을 통해 노드들이 직접 연결되어 데이터를 주고받음
- 긴급 구조, 긴급 회의, 군사적 상황 등에서 유용

보기: ㉠. Infrastructure Network ㉡. Firmware Network ㉢. Peer-to-Peer Network ㉣. Ad-hoc Network ㉤. Mesh Network ㉥. Sensor Network ㉦. Virtual Private Network

정답: ㉣. Ad-hoc Network

해설:  
**애드혹 네트워크(Ad-hoc Network)**는 라우터나 기지국 같은 **고정 인프라 없이**, 단말(노드)들이 서로 **직접 무선으로 연결**해 그때그때 임시로 구성하는 네트워크입니다.

- 'Ad-hoc'은 라틴어로 "이 목적을 위해 임시로"라는 뜻입니다.
- 중앙 관리 장치가 없어 빠르게 구성·해체할 수 있어, **긴급 상황(재난 구조, 군사 작전)**처럼 미리 깔아둔 인프라를 쓸 수 없는 곳에 적합합니다.

비교:

- Infrastructure Network: 기지국/AP 같은 고정 인프라를 거치는 일반적인 무선 네트워크.
- Ad-hoc Network: 인프라 없이 단말끼리 직접 연결.

키워드 "고정 인프라 없음 + 임시 구성 + 노드 직접 연결 = Ad-hoc"입니다.